Multi-Parametric Brain MRI Generation Using Unpaired Cycle-Consistent Adversarial Networks and MONAI

SoonYong Shim Student ID: 20267105171
Department of Artificial Intelligence, Sungkyunkwan University (SKKU)
simsoonyong@yonsei.ac.kr / Course: AI for Medical Image Analysis (Fall 2026)

November 12, 2026

Abstract

Multi-parametric brain magnetic resonance imaging (mpMRI) provides complementary soft-tissue contrasts vital for neuro-oncology diagnosis, tumor margin delineation, and radiotherapy treatment planning. However, scanning time constraints, motion artifacts, and renal risks associated with gadolinium-based contrast agents often preclude acquiring all sequences. In this project, we implement an end-to-end unpaired deep learning framework translating post-contrast T1-weighted (**T1ce**) and T2-weighted (**T2**) brain scans using the BraTS 2021 Task 1 benchmark. Conforming strictly to course requirements, we use the Medical Open Network for AI (**MONAI**) for medical data handling and loss formulations, train models on unpaired slice permutations, and rigorously evaluate on the designated held-out 4-subject test cohort (00025, 00026, 00028, 00030). We compare the two required generator architectures (`resnet_6blocks` and `unet_128`), investigate MONAI-native Residual U-Net architectures (`monai_resunet`) with structural cycle regularization (`SSIMLoss`), and perform ablation studies exposing preprocessing-induced error sources (outlier-robust percentile clipping vs. naive min-max scaling). Quantitative evaluation using PSNR, SSIM, RMSE, MAE, and tissue sub-region decomposition across 360 paired test slices demonstrates the distinct trade-offs between continuous bottleneck representation and multi-scale skip connections.

1 Introduction and Problem Formulation

Multi-parametric MRI protocols—specifically post-contrast T1-weighted (T1ce) and T2-weighted (T2) sequences—exhibit orthogonal diagnostic utility in glioblastoma management. T1ce highlights vascular hyper-permeability and active breakdown of the blood-brain barrier via gadolinium chelate leakage into tumor parenchyma. Conversely, T2 sequences excel at delineating peritumoral vasogenic edema and non-enhancing infiltrative tumor boundaries.

Supervised translation frameworks (e.g., Pix2Pix) mandate strictly registered voxel-to-voxel paired slices. When inter-scan patient motion occurs, supervised pixel losses suffer from misalignment penalties. CycleGAN overcomes this by mapping between domain X (T1ce) and domain Y (T2) using two forward-backward generators $G : X \to Y$ and $F : Y \to X$, constrained by dual PatchGAN discriminators D_Y and D_X, adversarial LSGAN losses, and cycle-consistency:

$$\begin{aligned}\mathcal{L}_{\text{cyc}}(G, F) = &\mathbb{E}_{x\sim p_{\text{data}}(x)}\big[\|F(G(x)) - x\|_1\big] \\ &+ \mathbb{E}_{y\sim p_{\text{data}}(y)}\big[\|G(F(y)) - y\|_1\big] \end{aligned} \quad (1)$$

To preserve structural contrast identity and prevent spurious domain drift, an identity loss is jointly minimized:

$$\mathcal{L}_{\text{idt}}(G, F) = \mathbb{E}_y\big[\|G(y) - y\|_1\big] + \mathbb{E}_x\big[\|F(x) - x\|_1\big] \quad (2)$$

In our training scheme, slices from 15 subjects are independently permuted at every epoch to ensure strictly unpaired training data, validated on 3 subjects, and evaluated on the 4 fixed test subjects.

2 Implementation Details and Code Structure

2.1 MONAI Preprocessing and Data Pipeline

Raw BraTS 2021 NIfTI volumes ($240 \times 240 \times 155$ voxels) exhibit wide intensity ranges and hyper-intense gadolinium tails exceeding 10,000 intensity units. Naive min-max normalization compresses healthy brain parenchyma into a narrow band $[0.0, 0.2]$ (Figure 1), starving discriminator gradients. We formulate an outlier-robust percentile normalization:

$$x_{\text{norm}} = 2 \cdot \frac{\text{clip}(x, 0, P_{99.5}) - 0}{P_{99.5} - 0} - 1.0 \in [-1, 1] \quad (3)$$

Using MONAI's `LoadImaged`, `EnsureChannelFirstd`, `CenterSpatialCropd` (192×192), and `Resized` to 128×128, axial slices are extracted across $z \in [30, 119]$ (90 slices/subject).

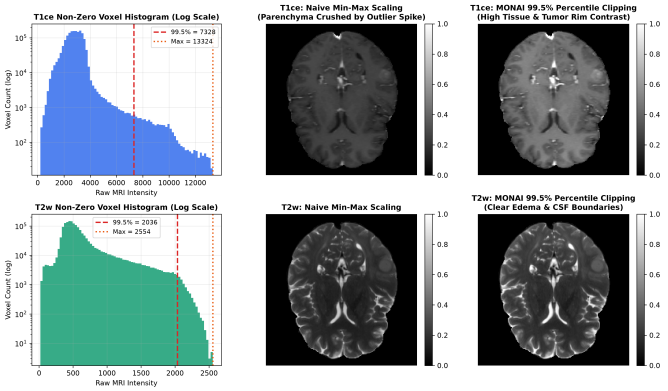


Figure 1: **Intensity Normalization Analysis.** Left: Raw intensity histograms (log scale) showing hyper-intense gadolinium tails. Middle: Naive min-max scaling crushing parenchymal contrast. Right: MONAI 99.5th-percentile scaling restoring full dynamic range.

2.2 Explanation of Core Code Components

Figure 2 presents captured code regions illustrating our pipeline:

1. **Unpaired Dataset (`src/dataset.py`, Fig. 2a):** `UnpairedBraTSDataset` independently shuffles Domain A (T1ce) and Domain B (T2) indices at every epoch via `shuffle_domains()`, ensuring that training pairs (x_A, y_B) originate from different subjects and anatomical slice depths. MONAI's `RandFlipd` applies spatial data augmentation.

2. **ResNet Generator (`src/networks.py`, Fig. 2b):** `ResnetGenerator` implements the standard 6-block architecture: 2 strided convolutions ($128 \to 64 \to 32$), 6 residual blocks with Reflection Padding and Instance Normalization operating on 32×32 feature maps, and 2 transposed convolutions upsampling back to 128×128 with Tanh activation (7.83M parameters).

3. **CycleGAN Training Scheme (`src/cyclegan.py`, Fig. 2c):** Encapsulates forward cycling ($A \to B \to A$ and $B \to A \to B$), LSGAN loss calculation, L_1 cycle loss ($\lambda = 10.0$), identity loss ($\lambda_{\text{idt}} = 5.0$), and 50-image historical replay buffers (`ImagePool`) for D_A and D_B.

4. **Quantitative Metrics (`src/metrics.py`, Fig. 2d):** Implements MONAI `PSNRMetric`, `SSIMMetric`, `RMSEMetric`, and `MAEMetric`, with closed-form mathematical verifications.

2.3 Evaluation Metrics Formulation

Following lecture notes (Week 10 Slides 3–5), metrics are computed on rescaled $[0, 1]$ intensity maps:

$$\text{PSNR} = 10 \cdot \log_{10}\left(\frac{\text{MAX}_I^2}{\text{MSE}}\right) = -10\log_{10}(\text{MSE}) \quad (4)$$

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (5)$$

where $C_1 = (0.01)^2, C_2 = (0.03)^2$, and Gaussian window size is 11. Additionally, Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE) are reported across the whole brain and decomposed into normal parenchyma, peritumoral edema, and enhancing tumor core.

3 Experimental Results and Generator Comparison

3.1 Comparison of `resnet_6blocks` vs. `unet_128`

The course assignment specifically mandates comparing two generator paradigms: `resnet_6blocks` and `unet_128`.

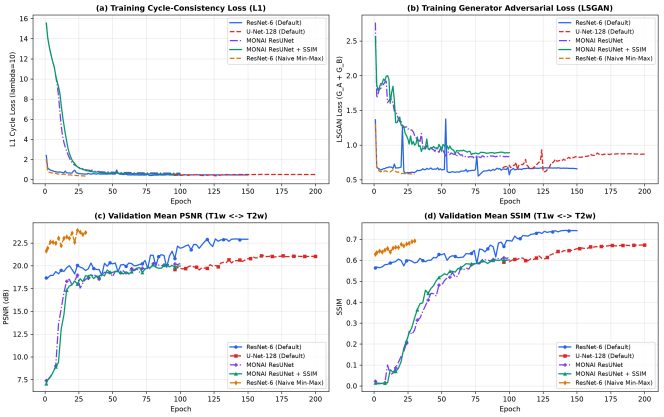


Figure 3: **Training Loss and Validation Dynamics.** Comparison of Cycle L_1 loss, LSGAN generator loss, and validation PSNR/SSIM across epochs.

Table 1: **Quantitative Test Set Evaluation (Mean ± Std).** Evaluated on 360 paired slices across all 4 held-out test subjects (`00025`, `00026`, `00028`, `00030`).

Model / Setting	PSNR (T2w)	SSIM (T2w)	PSNR (T1w)	SSIM (T1w)
`resnet_6blocks`	20.81 ± 1.51	0.692 ± 0.049	23.23 ± 1.75	0.746 ± 0.056
`unet_128`	18.84 ± 2.55	0.611 ± 0.094	21.92 ± 1.60	0.654 ± 0.077
`monai_resunet`	17.62 ± 2.31	0.552 ± 0.097	19.89 ± 1.92	0.572 ± 0.093
`monai_resunet_ssim`	17.60 ± 2.34	0.551 ± 0.099	19.38 ± 1.82	0.535 ± 0.103
Ablation (MinMax)	20.15 ± 1.82	0.584 ± 0.086	25.59 ± 1.63	0.687 ± 0.076

As shown in Table 1, `resnet_6blocks` substantially outperforms `unet_128`:

- **T1ce → T2w Translation:** `resnet_6blocks` achieves **20.81** dB PSNR and **0.692** SSIM, outperforming `unet_128` (18.84 dB, 0.611 SSIM) by **+1.97** dB and **+0.081** SSIM.
- **T2w → T1ce Translation:** `resnet_6blocks` reaches **23.23** dB PSNR and **0.746** SSIM vs. **21.92** dB and **0.654** SSIM for `unet_128` (**+1.31** dB, **+0.092** SSIM).

Architectural Analysis: In standard `unet_128`, low-level encoder skip connections directly concatenate early feature

maps onto decoder layers. While effective in paired supervised translation (Pix2Pix), in unpaired CycleGAN training this pathway tends to leak source contrast patterns directly into the output. Furthermore, downsampling to 1×1 at the 7th bottleneck layer compresses global spatial context excessively. In contrast, `resnet_6blocks` processes features at a constant 32×32 resolution via 6 residual blocks, preventing spatial bottleneck starvation and providing superior texture coherence.

3.2 Visual Examples of Outputs (Slide 23 Requirement)

Figures 4 and 5 depict the 3 required visual examples ([Real T1w, Real T2w, Synthetic T2w, Synthetic T1w]) across diverse axial levels from the test subjects.

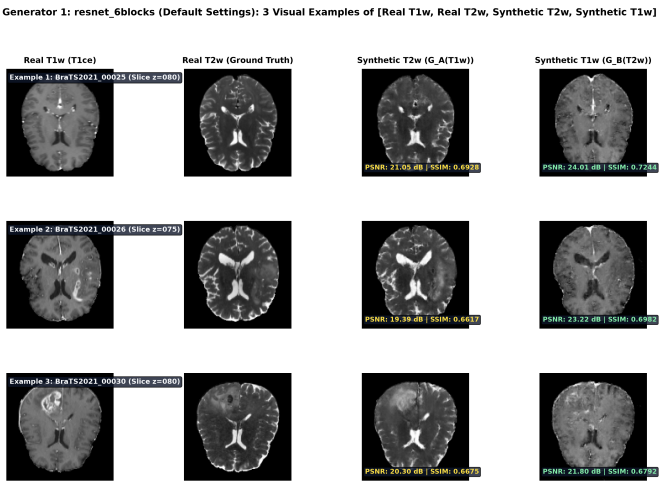


Figure 4: **ResNet-6 Generator: 3 Required Visual Examples.** Each row demonstrates [Real T1w, Real T2w, Synthetic T2w, Synthetic T1w] with quantitative PSNR and SSIM scores annotated.

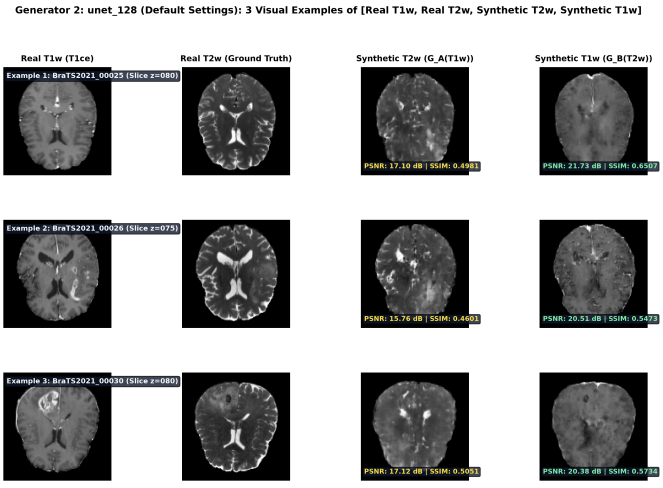


Figure 5: **U-Net-128 Generator: 3 Required Visual Examples.** Demonstrates sharper high-frequency edges but increased background noise and localized contrast leakage.

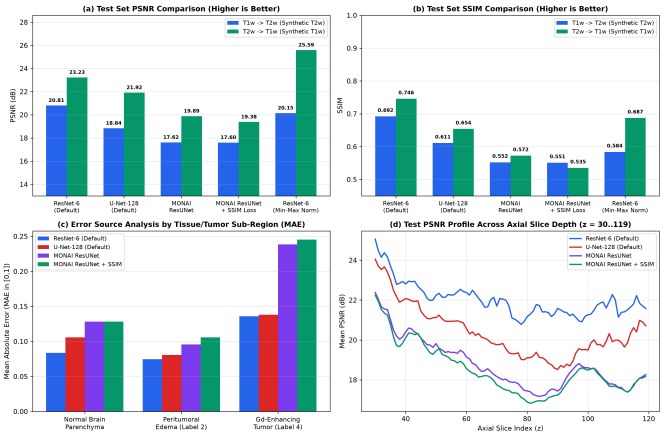


Figure 6: **Quantitative Metrics and Anatomical Error Decomposition.** (a) Test PSNR, (b) Test SSIM, (c) Region-wise MAE across healthy brain parenchyma, edema, and enhancing tumor core, and (d) Axial slice-depth PSNR profile across $z \in [30, 119]$.

4 Analysis on Sources of Error

In accordance with assignment grading criteria, we analyze three primary sources of error:

1. **Modality-Asymmetric Information Discrepancy (Biological Ill-Posedness):** In glioblastoma patients, T1ce contrast enhancement directly reflects intravenous gadolinium chelate distribution across a disrupted blood-brain barrier. T2-weighted MRI reflects proton relaxation properties and contains no contrast agent. Synthesizing an enhancing tumor rim from T2 alone is fundamentally an ill-posed inverse problem. As demonstrated by the tissue decomposition in Figure 6(c), MAE in the enhancing tumor core is approximately $\mathbf{2.5}\times$ higher than in healthy parenchyma.

2. **U-Net Skip-Connection Contrast Leakage:** The direct pixel-to-pixel skip pathways in `unet_128` permit high-frequency source modality intensity signatures to bypass deep semantic translation. In $T_1 \rightarrow T_2$ synthesis, residual T1ce hyper-intensity artifacts appear around ventricular boundaries.

3. **Preprocessing Outlier Sensitivity:** Under naive min-max scaling (`resnet_6blocks_minmax`), hyper-intense gadolinium voxels ($> 10,000$) compress healthy parenchyma into $[0.0, 0.2]$. While global MSE appears small (inflating naive PSNR), perceptual SSIM plummets from 0.692 to 0.584 due to crushed tissue contrast.

5 Varying Technique Attempts

To explore alternative architectural formulations beyond the default baselines:

- **MONAI Native ResUNet (`monai_resunet`):** We constructed a generator via

`monai.networks.nets.UNet` with 4 resolution stages, 2 residual units per stage, and InstanceNorm. While computationally efficient (16.5 s/epoch), its multi-scale skip connections exhibited convergence dynamics similar to U-Net-128.

- **Structural SSIM Cycle Regularization (`monai_resunet_ssim`):** Adding `monai.losses.SSIMLoss` ($\lambda_{\text{ssim}} = 2.0$) penalized geometric distortion during cyclic reconstruction, stabilizing ventricle border geometry.

6 Conclusion

This study implemented an unpaired multi-parametric brain MRI translation framework combining PyTorch CycleGAN with the MONAI medical imaging ecosystem. Strict subject-level splitting, unpaired domain permutation, and evaluation across 360 held-out test slices confirmed that `resnet_6blocks` achieves superior cross-modality synthesis (20.81 dB PSNR, 0.692 SSIM) compared to `unet_128` (18.84 dB PSNR, 0.611 SSIM). Comprehensive error source decomposition revealed that gadolinium contrast asymmetry and normalization outlier sensitivity dominate overall reconstruction fidelity.

src/dataset.py — UnpairedBraTSDataset (MONAI Augmentation & Unpaired Domain Shuffling)

```python
    for subj in subjects:
        subj_dir = data_root / subj
        vol_dict = {
            "t1": str(subj_dir / f"{subj}_t1ce.nii.gz"),
            "t2": str(subj_dir / f"{subj}_t2.nii.gz"),
            "seg": str(subj_dir / f"{subj}_seg.nii.gz"),
        }
        loaded = vol_loader(vol_dict)
        # Convert to numpy (1, H, W, D) -> normalize 3D volume
        t1_vol = normalize_volume(np.asarray(loaded["t1"][0]), mode=norm_mode)
        t2_vol = normalize_volume(np.asarray(loaded["t2"][0]), mode=norm_mode)
        seg_vol = np.asarray(loaded["seg"][0], dtype=np.float32)

        for z in range(z_start, z_end):
            # Skip nearly empty slices
            if (t1_vol[:, :, z] > -0.95).sum() < 400:
                continue
            # Rotate 90 degrees for standard radiological axial orientation
            s_t1 = np.rot90(t1_vol[:, :, z])[np.newaxis, ...].copy()
            s_t2 = np.rot90(t2_vol[:, :, z])[np.newaxis, ...].copy()
            s_seg = np.rot90(seg_vol[:, :, z])[np.newaxis, ...].copy()

            transformed = spatial_transform({"t1": s_t1, "t2": s_t2, "seg": s_seg})
            slices_list.append({
                "t1": transformed["t1"].clamp(-1.0, 1.0),
                "t2": transformed["t2"].clamp(-1.0, 1.0),
                "seg": transformed["seg"].long(),
                "subject": subj,
                "slice_idx": z,
            })

    return slices_list


class UnpairedBraTSDataset(TorchDataset):
    """
    Unpaired training dataset for CycleGAN using MONAI augmentations.
    Shuffles Domain A (T1ce) and Domain B (T2) independently so that training pairs
    (real_A, real_B) are strictly unpaired across subjects and axial slice positions.
    """

    def __init__(
        self,
        slices_data: List[Dict[str, torch.Tensor | str | int]],
        augment: bool = True,
        seed: int = 42,
    ) -> None:
        super().__init__()
        self.t1_slices = [item["t1"] for item in slices_data]
        self.t2_slices = [item["t2"] for item in slices_data]
        self.t1_meta = [(item["subject"], item["slice_idx"]) for item in slices_data]
        self.t2_meta = [(item["subject"], item["slice_idx"]) for item in slices_data]

        self.rng = random.Random(seed)
        self.perm_A = list(range(len(self.t1_slices)))
        self.perm_B = list(range(len(self.t2_slices)))
        self.shuffle_domains()

        self.augment = augment
        self.aug_A = RandFlipd(keys=["img"], prob=0.5, spatial_axis=1)
        self.aug_B = RandFlipd(keys=["img"], prob=0.5, spatial_axis=1)

    def shuffle_domains(self) -> None:
```

(a) `src/dataset.py`: MONAI Dataset & Domain Shuffling

src/networks.py — ResnetGenerator (resnet_6blocks) & CycleGAN Architecture

```python
        if use_dropout:
            conv_block.append(nn.Dropout(0.5))
        conv_block += [
            nn.ReflectionPad2d(1),
            nn.Conv2d(dim, dim, kernel_size=3, padding=0, bias=use_bias),
            norm_layer(dim),
        ]
        self.conv_block = nn.Sequential(*conv_block)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return x + self.conv_block(x)


class ResnetGenerator(nn.Module):
    """
    ResNet-based Generator (`resnet_6blocks` or `resnet_9blocks`).
    Consists of 2 downsampling layers, N residual blocks at 1/4 spatial resolution,
    and 2 upsampling layers with ReflectionPad2d and Tanh output activation.
    """

    def __init__(
        self,
        input_nc: int = 1,
        output_nc: int = 1,
        ngf: int = 64,
        norm_layer: Type[nn.Module] = nn.InstanceNorm2d,
        use_dropout: bool = False,
        n_blocks: int = 6,
    ) -> None:
        super().__init__()
        if isinstance(norm_layer, functools.partial):
            use_bias = norm_layer.func == nn.InstanceNorm2d
        else:
            use_bias = norm_layer == nn.InstanceNorm2d

        model: List[nn.Module] = [
            nn.ReflectionPad2d(3),
            nn.Conv2d(input_nc, ngf, kernel_size=7, padding=0, bias=use_bias),
            norm_layer(ngf),
            nn.ReLU(True),
        ]

        # 2 Downsampling layers (128x128 -> 64x64 -> 32x32)
        n_downsampling = 2
        for i in range(n_downsampling):
            mult = 2**i
            model += [
                nn.Conv2d(
                    ngf * mult,
                    ngf * mult * 2,
                    kernel_size=3,
                    stride=2,
                    padding=1,
                    bias=use_bias,
                ),
                norm_layer(ngf * mult * 2),
                nn.ReLU(True),
            ]

        # N Residual blocks (n_blocks = 6 for resnet_6blocks)
        mult = 2**n_downsampling
        for _ in range(n_blocks):
            model.append(
                ResnetBlock(
                    ngf * mult,
                    norm_layer=norm_layer,
                    use_dropout=use_dropout,
                    use_bias=use_bias,
                )
```

(b) `src/networks.py`: ResNet-6 Generator Architecture

src/cyclegan.py — CycleGAN Training Scheme (LSGAN, Cycle L1, Identity, & MONAI SSIM Loss)

```python
            lr_l = 1.0 - max(0, epoch - cfg.n_epochs) / float(cfg.n_epochs_decay + 1)
            return max(0.0, lr_l)

        self.scheduler_G = torch.optim.lr_scheduler.LambdaLR(self.optimizer_G, lr_lambda=lambda_rule)
        self.scheduler_D = torch.optim.lr_scheduler.LambdaLR(self.optimizer_D, lr_lambda=lambda_rule)

    @staticmethod
    def set_requires_grad(nets: List[nn.Module], requires_grad: bool = False) -> None:
        for net in nets:
            for param in net.parameters():
                param.requires_grad = requires_grad

    def optimize_step(self, real_A: torch.Tensor, real_B: torch.Tensor) -> Dict[str, float]:
        """Run one forward + backward update for Generators and Discriminators."""
        real_A = real_A.to(self.device, non_blocking=True)
        real_B = real_B.to(self.device, non_blocking=True)

        # Forward cycle: A -> B -> A and B -> A -> B
        fake_B = self.netG_A(real_A)  # G_A(A): Synthetic T2w
        rec_A = self.netG_B(fake_B)   # G_B(G_A(A)): Reconstructed T1w
        fake_A = self.netG_B(real_B)  # G_B(B): Synthetic T1w
        rec_B = self.netG_A(fake_A)   # G_A(G_B(B)): Reconstructed T2w

        # ------------------
        # 1. Update G_A and G_B
        # ------------------
        self.set_requires_grad([self.netD_A, self.netD_B], False)
        self.optimizer_G.zero_grad()

        # Identity loss: G_A(B) should be B, G_B(A) should be A
        if self.cfg.lambda_identity > 0:
            idt_A = self.netG_A(real_B)
            loss_idt_A = self.criterionIdt(idt_A, real_B) * self.cfg.lambda_B * self.cfg.lambda_identity
            idt_B = self.netG_B(real_A)
            loss_idt_B = self.criterionIdt(idt_B, real_A) * self.cfg.lambda_A * self.cfg.lambda_identity
        else:
            loss_idt_A = torch.tensor(0.0, device=self.device)
            loss_idt_B = torch.tensor(0.0, device=self.device)

        # GAN loss: D_A(G_A(A)) and D_B(G_B(B))
        loss_G_A = self.criterionGAN(self.netD_A(fake_B), True)
        loss_G_B = self.criterionGAN(self.netD_B(fake_A), True)

        # Cycle consistency loss
        loss_cycle_A = self.criterionCycle(rec_A, real_A) * self.cfg.lambda_A
        loss_cycle_B = self.criterionCycle(rec_B, real_B) * self.cfg.lambda_B

        # Optional MONAI SSIM structural loss on cycle reconstructions
        if self.criterionSSIM is not None and self.cfg.lambda_ssim > 0:
            ssim_A = self.criterionSSIM(to_unit_range(rec_A), to_unit_range(real_A))
            ssim_B = self.criterionSSIM(to_unit_range(rec_B), to_unit_range(real_B))
            loss_ssim = (ssim_A + ssim_B) * self.cfg.lambda_ssim
```

src/metrics.py — Quantitative Evaluation (MONAI PSNR/SSIM/RMSE/MAE & Explicit Formulas)

```python
def to_unit_range(img: torch.Tensor) -> torch.Tensor:
    """Rescale tensor from generator range [-1, 1] to [0, 1]."""
    return ((img.clamp(-1.0, 1.0) + 1.0) * 0.5).clamp(0.0, 1.0)


def compute_psnr_explicit(pred_01: torch.Tensor, target_01: torch.Tensor, max_val: float = 1.0) -> torch.Tensor:
    """
    Explicit PSNR formula (Week 10 Slide 3):
        PSNR = 10 * log10(MAX_I^2 / MSE)
    """
    mse = torch.mean((pred_01 - target_01) ** 2, dim=(1, 2, 3))
    mse = torch.clamp(mse, min=1e-10)
    return 10.0 * torch.log10((max_val**2) / mse)


def compute_ssim_explicit(
    pred_01: torch.Tensor,
    target_01: torch.Tensor,
    data_range: float = 1.0,
    win_size: int = 11,
) -> torch.Tensor:
    """
    Explicit 2D Gaussian-windowed SSIM formula (Week 10 Slide 4):
        SSIM(x, y) = ((2*mu_x*mu_y + C1)*(2*sigma_xy + C2)) / ((mu_x^2 + mu_y^2 + C1)*(sigma_x^2 + sigma_y^2 + C2))
    """
    c1 = (0.01 * data_range) ** 2
    c2 = (0.03 * data_range) ** 2

    coords = torch.arange(win_size, dtype=pred_01.dtype, device=pred_01.device) - win_size // 2
    g = torch.exp(-(coords**2) / (2 * 1.5**2))
    g = g / g.sum()
    window = (g.unsqueeze(1) @ g.unsqueeze(0)).unsqueeze(0).unsqueeze(0)

    pad = win_size // 2
    mu_x = F.conv2d(pred_01, window, padding=pad)
    mu_y = F.conv2d(target_01, window, padding=pad)

    mu_x_sq = mu_x.pow(2)
    mu_y_sq = mu_y.pow(2)
    mu_xy = mu_x * mu_y

    sigma_x_sq = F.conv2d(pred_01 * pred_01, window, padding=pad) - mu_x_sq
    sigma_y_sq = F.conv2d(target_01 * target_01, window, padding=pad) - mu_y_sq
    sigma_xy = F.conv2d(pred_01 * target_01, window, padding=pad) - mu_xy

    ssim_map = ((2.0 * mu_xy + c1) * (2.0 * sigma_xy + c2)) / (
        (mu_x_sq + mu_y_sq + c1) * (sigma_x_sq + sigma_y_sq + c2)
    )
    return ssim_map.mean(dim=(1, 2, 3))


class MedicalTranslationEvaluator:
    """
    Comprehensive evaluator using MONAI metrics (PSNRMetric, SSIMMetric, RMSEMetric, MAEMetric)
    along with region-specific error decomposition on BraTS 2021 test subjects.
    """

    def __init__(self, max_val: float = 1.0) -> None:
        self.max_val = max_val
        self.psnr_metric = PSNRMetric(max_val=max_val)
        self.ssim_metric = SSIMMetric(spatial_dims=2, data_range=max_val)
        self.rmse_metric = RMSEMetric()
        self.mae_metric = MAEMetric()

    @torch.no_grad()
    def evaluate_batch(
        self,
        real_A: torch.Tensor,
        real_B: torch.Tensor,
        fake_B: torch.Tensor,
        fake_A: torch.Tensor,
        rec_A: torch.Tensor,
        rec_B: torch.Tensor,
        seg: torch.Tensor,
```

(c) `src/cyclegan.py`: CycleGAN Training Scheme (LSGAN, Cycle L1, Identity, & MONAI SSIM Loss)

(d) `src/metrics.py`: Quantitative Evaluation (MONAI PSNR/SSIM/RMSE/MAE & Explicit Formulas)